

Similarity-Based Assessment of Computational Reproducibility in Jupyter Notebooks

A S M Shahadat Hossain
University of Missouri
Columbia, Missouri, USA
ahr8v@missouri.edu

David Koop
Northern Illinois University
DeKalb, Illinois, USA
dakoop@niu.edu

Colin Brown
Northern Illinois University
DeKalb, Illinois, USA
colinjbrown@niu.edu

Tanu Malik
University of Missouri
Columbia, Missouri, USA
tanu@missouri.edu

Abstract

Computational reproducibility refers to obtaining consistent results when rerunning an experiment. Jupyter Notebook, a web-based computational notebook application, facilitates the execution, publication, and sharing of computational experiments along with their results. However, rerunning a Jupyter Notebook may not always produce identical results due to factors such as randomness, changes in library versions, or variations in the computational environment. This paper introduces the *Similarity-based Reproducibility Index (SRI)* – a metric for assessing the reproducibility of results in Jupyter Notebooks. SRI employs novel methods based on similarity metrics specific to different types of Python objects to compare rerun outputs against original outputs. For each cell that produces an output in a rerun notebook, SRI reports a quantitative score in the range [0, 1], along with qualitative insights to support reproducibility assessment. The paper also presents an evaluation in which the proposed metric is applied to a set of 100 Jupyter Notebooks, demonstrating how various similarity metrics can be leveraged to quantify computational reproducibility.

CCS Concepts

• **General and reference** → **Metrics**; *Verification*; **Evaluation**; *Validation*; • **Information systems** → **Computing platforms**; • **Computer systems organization** → *Reliability*; • **Software and its engineering** → *Software libraries and repositories*.

Keywords

Computational Reproducibility, Jupyter Notebooks, Reproducibility Assessment, Similarity Metrics, Reproducibility Metric, SRI, Output Comparison.

ACM Reference Format:

A S M Shahadat Hossain, Colin Brown, David Koop, and Tanu Malik. 2025. Similarity-Based Assessment of Computational Reproducibility in Jupyter Notebooks. In *ACM Conference on Reproducibility and Replicability (ACM REP '25)*, July 29–31, 2025, Vancouver, BC, Canada. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3736731.3746159>

1 Introduction

In recent years, computational reproducibility has emerged as a major concern across multiple research communities. It has drawn increasing attention since the 2019 publication of the National Academies of Sciences, Engineering, and Medicine report titled *Reproducibility and Replicability in Science* [21]. Improved reproducibility not only enhances scientific trust but also saves energy, time, and financial resources. As a result, disciplines beyond computer science, including geoscience [35], management science [10], and the social sciences [20], have recognized the need to foster a culture of reproducibility.

Ideally, rerunning a scientific experiment is expected to produce outcomes identical to the original results. However, this expectation often fails due to a variety of factors, such as changes in environmental conditions, recording methods, or even units of measurement. Computational experiments face similar challenges. Even when the same code is executed with the same data in the same environment, the results may differ, sometimes subtly, sometimes significantly enough to affect interpretation. Yet traditional approaches to assessing this first form of reproducibility—also known as repeatability—typically treat it as a binary outcome, offering no means to distinguish between minor deviations and substantial inconsistencies. This limitation highlights the need for a more refined metric that can capture the extent of consistency in rerun outputs, rather than labeling them simply as ‘identical’ or ‘different’.

Jupyter Notebook is a popular platform for interactive workflows, enabling users to divide code into modular execution blocks, called *cells*, which can be executed independently and in any order. While this flexibility facilitates exploratory analysis and iterative development, it does not guarantee consistent outputs across multiple re-executions. Output differences can arise from sources such as nondeterminism, changes in library versions, or differences in the underlying computing environment. Several prior works [1, 25–27, 33, 39] have investigated reproducibility in

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ACM REP '25, Vancouver, BC, Canada

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1958-5/25/07

<https://doi.org/10.1145/3736731.3746159>

Jupyter Notebooks through approaches such as detecting binary differences in cell outputs, identifying potential causes of those differences, and developing tools for containerized re-execution by tracking library and environment-level dependencies to mitigate inconsistent results. However, there remains a lack of a standard metric that can go beyond binary comparison to quantify how different the rerun outputs are from the originals.

In this paper, we propose the *Similarity-based Reproducibility Index (SRI)*, a novel metric designed to move beyond traditional binary difference detection by not only identifying when outputs differ, but also quantifying how different they are. SRI combines quantitative and qualitative assessments of reproducibility in Jupyter Notebooks: the quantitative component assigns a reproducibility score in the range $[0, 1]$ using similarity metrics tailored to different output types, while the qualitative component provides semantic insights into the structure and characteristics of the outputs. This approach enables a more granular and informative evaluation of reproducibility than strict equality checks allow.

SRI is particularly useful in collaborative scenarios where Jupyter Notebooks are shared and rerun across different machines, environments, or even distributed computing platforms. In such settings, collaborators often work in slightly different computing contexts, and notebooks may be re-executed on varied hardware configurations, clusters, or cloud-based systems. These differences can introduce subtle output changes that traditional binary checks may fail to capture. By measuring the degree of similarity between rerun and original outputs, SRI helps users assess whether intermediate or final results remain consistent. This makes it valuable not only for researchers and reviewers, but also for organizations and infrastructure providers that operate outside tightly controlled containerized environments and seek to evaluate the stability and reproducibility of computational results over time.

We apply SRI at the cell level by comparing outputs of different types of Python objects extracted from the original and the rerun notebooks. The cell-level scores are then aggregated into a notebook-level SRI score. Both levels of results are stored in a single JSON (JavaScript Object Notation) file per notebook, which facilitates portability, analysis, and integration with other tools. We evaluate the proposed metric on a set of 100 Jupyter Notebooks. The results show that SRI can detect partial reproducibility in outputs that traditional methods classify as entirely non-reproduced. This finer-grained assessment makes SRI a more meaningful and practical metric to assess reproducibility in real-world notebook executions.

The rest of the paper is organized as follows: Section 2 defines key terminology with illustrative examples. Section 3 presents the methodology behind the proposed metric. Section 4 evaluates the metric by applying it to a set of Jupyter Notebooks. Section 5 reviews related work, and Section 6 concludes the paper.

2 Definitions

2.1 Cells in Jupyter Notebooks

A *cell* is the fundamental building block and modular unit of a Jupyter Notebook. It enables modularity by allowing users to decompose a computational workflow into discrete, manageable

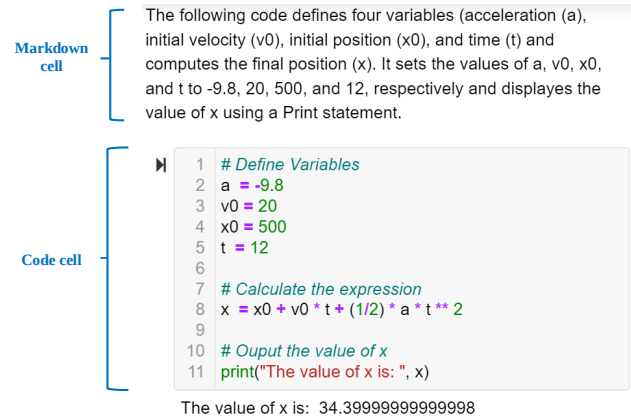


Figure 1: Code Cell and Markdown Cell in a Jupyter Notebook.

steps that can be executed independently or in sequence. Jupyter Notebooks support three types of cells:

- **Code cells:** Being the most prominent type of cell in notebooks, a code cell is used to write code. Code cells automatically support smart code formatting, such as syntax highlighting and indentations. When this type of cell is executed, the code written in it is sent to the kernel responsible for running the code. If the code generates any results, then the result is displayed as the output of the respective code cell.
- **Markdown cells:** This type of cell facilitates documentation of the code. The notebook author can use markdown cells to describe the code or the computational processes using natural texts, rich text, and even markup languages. The cells support text formatting using HTML and standard LaTeX notations.
- **Raw cells:** This type of cell is the least commonly used one in notebooks. Raw cells are used to write raw code for the notebook to be rendered or converted into different formats, such as HTML or LaTeX. Raw cells are not evaluated by notebooks.

Figure 1 presents an example of a code cell and a markdown cell in a Jupyter Notebook.

2.2 Types of Code Cell Outputs

Jupyter Notebook supports various types of outputs. Among them, the major types of outputs observed as the code cell outputs are as follows:

- **stream outputs:** These outputs are typically generated using the print statement or other standard output functions. The most common MIME types associated with stream outputs are text/plain and text/html.
- **display_data outputs:** In the JSON structure of a notebook, display_data represents the rich type display outputs such as images. Among different formats of images, image/png is the most frequent one observed. Besides,

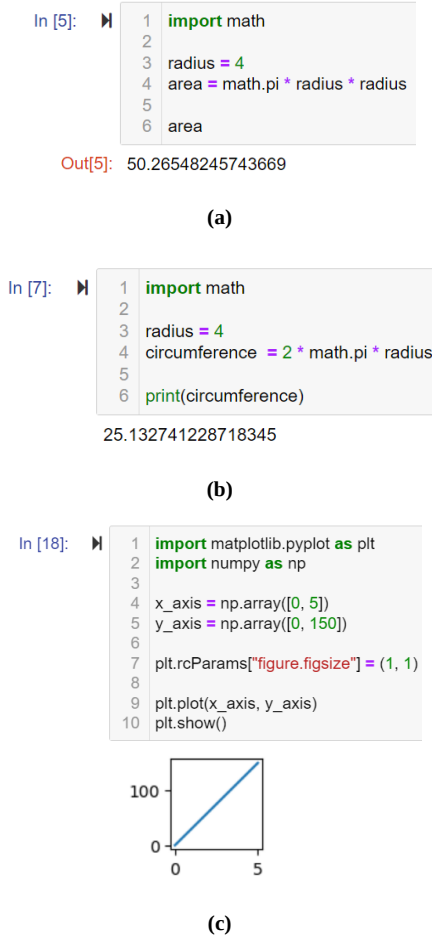


Figure 2: Different Types of Code Cell Outputs in a Jupyter Notebook (a) execute_result (b) stream (c) display_data.

image/jpeg and image/svg are some of the other MIME types of display_data outputs.

- **execute_result outputs:** Jupyter Notebook offers a useful feature of writing the name of an object at the end of a code cell to display it without the need to use any print statement. The outputs displayed in such cases are generally stored as execute_result outputs in the notebook's JSON.
- **error outputs:** This type of output represents runtime errors due to the failed execution of the code in a code cell.

Figure 2 illustrates major types of code cell outputs in a Jupyter Notebook.

3 Methodology

The Similarity-based Reproducibility Index (SRI) is the proposed metric to approximate reproducibility in Jupyter Notebooks. As the name suggests, this metric is built on the premise that higher similarity between two outputs corresponds to higher reproducibility, even when the outputs are not exactly identical.

Algorithm 1 SRI for Jupyter Notebooks

```

1: Input: original_notebook, rerun_notebook
2: Output: JSON structure of notebook-level SRI
3: for each code cell generating outputs do
4:   Parse the cell outputs and determine the output_type
5:   if output_type == stream then
6:     Apply method to compare two string outputs
7:   else if output_type == display_data then
8:     Apply method to compare two image outputs
9:   else if output_type == execute_result then
10:    Determine specific object_type of the outputs
11:    if object_type ∈ {int, float, list, tuple, dict, str,
12:      bool, numpy.ndarray, pandas.DataFrame, pandas.Series,
13:      dict_keys} then
14:      Apply corresponding methods to compare outputs
15:    else
16:      Apply method to compare two string outputs
17:    end if
18:  end for
19: Generate cell-level SRI
20: Generate notebook-level SRI from the cell-level SRIs

```

An SRI score for a notebook code cell that generates outputs consists of a quantitative reproducibility score in the range [0, 1] and several qualitative insights related to similarity. At the notebook level, SRI is represented as a JSON (JavaScript Object Notation) structure that includes the names of the original and rerun notebooks, a list of cell-level SRI results, and the average of all cell-level reproducibility scores. It also tracks the execution IDs of the corresponding cells in both versions of the notebook. If an output pair is compared using multiple methods (e.g., an output compared both as a string and as a list), the notebook-level SRI includes results for each comparison, along with both the original and the converted object types.

To apply SRI, we first parse different types of code cell outputs (as discussed in Section 2.2) from both the original notebooks and their rerun versions. For the execute_result outputs, we determine and store their specific object type in the cell metadata while rerunning the notebooks. After that, we apply different methods to the outputs based on the object type information. In contrast, the stream outputs are simply rendered as plain texts, typically generated using print statements, and thus lack metadata that can indicate their original object type. As a result, we cannot determine the exact underlying object type of stream outputs and evaluate their reproducibility using string similarity. Similarly, we apply image similarity metrics to the display_data outputs rendered as images. Algorithm 1 summarizes the entire methodology.

3.1 SRI for int or float Outputs

For int and float type outputs, the SRI score is set to 1 if the original and rerun outputs are found to be identical. Otherwise, the absolute and relative differences between them are calculated. The relative difference is calculated in the percentage of changes in

the rerun output with respect to the original output. Since a rerun *float* output may sometimes differ insignificantly from the original due to randomness or precision-related changes influenced by hardware, platform, or libraries, a tolerance threshold of $1e-09$ for the difference is considered. If the difference between two *float* values falls within this range, the SRI considers the result to be reproduced. If the difference exceeds this threshold and the values are neither equal nor sufficiently close, the SRI score is set to 0.

3.2 SRI for *list* or *tuple* Outputs

Since *list* and *tuple* in Python are both ordered and iterable sequences with the only difference in mutability (a *tuple*, unlike a *list*, can not be modified after its creation), the same SRI method can be used to evaluate reproducibility between two *list* or two *tuple* outputs. When such outputs are not exactly identical, the method looks further into the output characteristics from various similarity perspectives. For example, it checks if the original and the rerun output have the same or different lengths. It also checks if the sorted forms of the outputs would match each other. If the *list* or *tuple* contains only numeric values, i.e., all their elements are either *int* or *float*, the method tests if the outputs share the same maximum and minimum elements. It gives an indication of whether the numeric elements of the objects are in the same particular range. Additionally, it also reports the percentage of common distinct elements between two outputs, irrespective of their indices. Though it may not be a definitive assessment or reproducibility, it can facilitate the user's understanding of the nature of output changes, specifically in scenarios where the entities represented by the elements do not have any index-wise one-to-one relationship. For example, between two *list* outputs representing a *pandas* dataframe column names, the index-independent presence of elements is more important than their respective index positions. Finally, the SRI method determines the percentage of index-wise common elements between two outputs having the same length to use that as a metric to quantify the overall reproducibility. Thus, this method ensures a more precise reproducibility assessment between two *list* or *tuple* objects, unlike using the traditional reproducibility evaluation that would result in either 0 or 100% reproducibility.

3.3 SRI for *dict* Outputs

A Python *dict* is a collection of items consisting of key-value pairs. The changes in a rerun *dict* output can be observed as a change in the number of items and changes in keys and/or their associated values. The SRI checks how many keys from the original *dict* are present in the rerun *dict*. The overall reproducibility score of *dict* outputs is quantified as the percentage of identical items matching both the keys and the values between the original and the rerun outputs.

3.4 SRI for *str* Outputs

If two *str* outputs are not identical, the SRI method still considers the result as reproduced, given that they match after ignoring the white spaces or in a case-insensitive comparison. It also reports whether the original and the rerun *str* outputs are a substring of each other. This can provide useful insights in cases where one result is partially

present in another result, or the change in the rerun *str* output may not be semantically significant. Finally, SRI determines the similarity between two outputs by using the Jaro-Winkler algorithm [44]. This algorithm works based on the following equations and provides an edit distance-based string similarity score between 0 and 1. It then uses this score as the overall reproducibility score for the *str* outputs.

$$\text{Jaro similarity, } sim_j = \begin{cases} 0 & \text{if } m = 0 \\ \frac{1}{3} \left(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m} \right) & \text{otherwise} \end{cases} \quad (1)$$

where: $|s_i|$ is the length of the string s_i ; m is the number of matching characters; t is the number of transpositions.

$$\text{Jaro-Winkler similarity} = sim_j + lp(1 - sim_j) \quad (2)$$

where: l is the length of the common prefix at the start of the string up to a maximum of 4 characters; p is a constant scaling factor.

3.5 SRI for *NumPy* array Outputs

Numpy [13] is one of the most popular Python libraries. The fundamental object of this library, called *Numpy array*, provides faster computation than the traditional Python *lists*. The SRI method first compares the dimensions, sizes, and shapes of the rerun array with the original array. If the shapes of the arrays match, then it performs an element-wise comparison among the array elements. The percentage of elements matched between the two arrays is set as the reproducibility score in such cases. For the arrays with *float* elements, outputs differing in a particular small range ($1e-08$) are considered as reproduced. If the shapes between two array outputs are found to be different, then this method performs an index-independent comparison to report the percentage of elements that are common between the arrays, though their index positions may be different. While this comparison may add very little value in quantifying the reproducibility, there might be possible scenarios where this kind of insight would make the comparison more comprehensive and meaningful.

A large *Numpy* array output is often abbreviated in the notebook display. Ellipses (...) are used as placeholders for the deprecated elements across the array dimensions. How many elements of an array will be displayed depends on the parameter setting of the function called *numpy.set_printoptions* [22] in the particular environment where the notebook is running. The *edgeitems* parameter of this function sets the number of elements to be displayed across the array dimensions. While the default value of *edgeitems* is 3, it can be configured by passing a different value of this parameter to the function to display a different number of array elements.

According to the traditional reproducibility evaluation concept, two array outputs having different numbers of visible elements will never be considered reproducible, even though they are the same arrays, just with different representations. Since there is no known way to retrieve the invisible elements from the original array with ellipses, they can not be compared against the corresponding elements in the rerun array with different numbers of elements

```

original_array
array([[0.85888927, 0.37271115, 0.55512878, 0.95565655, 0.7366696 ],
       [0.81620514, 0.10108656, 0.92848807, 0.60910917, 0.59655344],
       [0.09178413, 0.34518624, 0.66275252, 0.44171349, 0.55148779],
       ...,
       [0.08199788, 0.0718493 , 0.07337724, 0.43259797, 0.35905958],
       [0.4937732 , 0.46680198, 0.03935498, 0.1182524 , 0.01818196],
       [0.19237866, 0.52641283, 0.4544607 , 0.83813735, 0.47002215]])

rerun_array
array([[0.85888927, 0.37271115, 0.55512878, 0.95565655, 0.7366696 ],
       [0.81620514, 0.10108656, 0.92848807, 0.60910917, 0.59655344],
       [0.09178413, 0.34518624, 0.66275252, 0.44171349, 0.55148779],
       ...,
       [0.70371249, 0.58940123, 0.04993276, 0.56179184, 0.76635847],
       [0.91090833, 0.09290995, 0.90252139, 0.46096041, 0.45201847],
       ...,
       [0.9215039 , 0.10276089, 0.47234306, 0.63929769, 0.93584964],
       [0.4573361 , 0.30017064, 0.61993774, 0.48971651, 0.23635369],
       [0.08199788, 0.0718493 , 0.07337724, 0.43259797, 0.35905958],
       [0.4937732 , 0.46680198, 0.03935498, 0.1182524 , 0.01818196],
       [0.19237866, 0.52641283, 0.4544607 , 0.83813735, 0.47002215]])

```

Figure 3: An Example of Comparing Two *Numpy* array Outputs with Ellipses in One Dimension.

```

original_array
array([[0.8589, 0.3727, 0.5551, ..., 0.3624, 0.4484, 0.7613],
       [0.343 , 0.7451, 0.8588, ..., 0.0983, 0.1926, 0.6298],
       [0.0841, 0.3242, 0.01 , ..., 0.4487, 0.8635, 0.7211]])

rerun_array
array([[0.8589, 0.3727, ..., 0.4484, 0.7613],
       [0.343 , 0.7451, ..., 0.1926, 0.6298],
       [0.0841, 0.3242, ..., 0.8635, 0.7211]])

```

Figure 4: Another Example of Comparing Two *Numpy* array Outputs with Ellipses in One Dimension.

displayed. Similarly, due to a lower number of *edgitems* settings in the rerun environment, it will display a smaller number of array elements to compare against the elements of an original array. Because of the mismatch between the number of visible elements in the two arrays, it will not be possible to calculate one-to-one similarity, and thus, it will result in zero reproducibility.

Because the SRI method prioritizes a comprehensive comparison between any two outputs, it separates the visible elements of the arrays after excluding the ellipses and trimming them based on the common specific indices of the visible elements. After that, a one-to-one element-wise comparison is performed to calculate reproducibility for two arrays of the same shape having the same number of visible elements. Figure 3 and Figure 4 show illustrations of this operation on two *arrays*. For arrays of different shapes with ellipses across two dimensions, the elements are split into four quadrants based on the positions of the ellipses, and the trimmed arrays are compared to each other, as shown in Figure 5.

For all three of the figures (Figures 3, 4, 5) showing *Numpy* array comparisons, according to the SRI method, we exclude the ellipses, remove the elements inside the red boxes, and compare the elements inside the boxes of same color and/or style. Since the original *array* outputs are exactly the same as the rerun *array* outputs, just with different representations, this method finds that they are fully reproduced. Thus, the proposed method facilitates a more comprehensive reproducibility assessment over the existing

```

original_array
array([[0.8589, 0.3727, 0.5551, ..., 0.3624, 0.4484, 0.7613],
       [0.343 , 0.7451, 0.8588, ..., 0.0983, 0.1926, 0.6298],
       [0.0841, 0.3242, 0.01 , ..., 0.4487, 0.8635, 0.7211],
       ...,
       [0.696 , 0.3467, 0.9734, ..., 0.4941, 0.5805, 0.0439],
       [0.6452, 0.6675, 0.0579, ..., 0.2764, 0.3034, 0.0114],
       [0.0573, 0.7879, 0.9685, ..., 0.6818, 0.1943, 0.7209]])

rerun_array
array([[0.8589, 0.3727, ..., 0.4484, 0.7613],
       [0.343 , 0.7451, ..., 0.1926, 0.6298],
       ...,
       [0.6452, 0.6675, ..., 0.3034, 0.0114],
       [0.0573, 0.7879, ..., 0.1943, 0.7209]])

```

Figure 5: An Example of Comparing Two *Numpy* array Outputs with Ellipses in Two Dimensions.

traditional ways to evaluate the reproducibility of *Numpy* array outputs.

3.6 SRI for *Pandas DataFrame* Outputs

Besides *Numpy*, *Pandas* [36] is another popular Python library. *DataFrames*, being the fundamental object of this library, are two-dimensional data structures used for tabular data. The SRI method uses BeautifulSoup [28] to parse the *DataFrames* rendered with *text/html* MIME type in notebooks. The column names and the indices are extracted in this process. After that, two *DataFrame* outputs are compared, and the similarity characteristics, such as the number of rows and the number of columns, are captured. The method also checks how many column names exactly match between two *DataFrames*. The possibility of the elements being different between two outputs, even after sharing common column names and indices, is also considered. That is why the method finally calculates the percentage of matched elements among all the elements from the matching columns and indices to determine the overall reproducibility. Figure 6 illustrates an example of applying the SRI method on two *DataFrame* outputs. As shown in the figure, data (elements inside the yellow boxes) located only in the cells sharing the same column names and the same indices are compared to calculate the reproducibility score. Since the SRI method provides qualitative insights besides the reproducibility scores, it also reports other similarity aspects of the outputs, such as whether they have the same number of rows and columns, as well as the percentage of column headers and indices matched.

3.7 SRI for *image* Outputs

Image outputs are a predominant type of result in many Jupyter Notebooks, especially in fields involving data visualization, machine learning, and scientific computing. Compared to other output types, assessing the reproducibility of image outputs presents unique challenges. Even small changes in rendering, resolution, color encoding, or metadata can alter the byte-level representation of an image, despite the visual information being nearly identical. Unlike textual or numeric outputs, where one can directly compare strings or values, visual outputs require perceptual evaluation to determine whether two images effectively communicate the same result.

original_dataframe					rerun_dataframe				
	file name	sift	ssim	sum		file name	sift	ssim	
0	08f2821...	0.051	0.227	0.278	0	08f2821...	0.0512	0.227	
1	0934162...	0.177	0.648	0.824	1	0934162...	0.1767	1.6475	
2	09aafcc...	0.29	0.832	1.122	2	09aafcc...	0.2899	1.8318	
3	09aafcc...	0.308	0.821	1.129	3	09aafcc...	0.3077	1.8213	
4	0a51afd...	0.316	0.848	1.165	4	0a51afd...	0.3164	1.8483	
...	...	...	...	...	...	...	...	...	...
354	0699d90...	0.618	0.827	1.445	195	6899df7...	0.0324	1.1667	
355	06e1bcb...	0.323	0.633	0.956	196	6899df7...	0.021	1.0471	
356	087b99d...	0.319	0.652	0.971	197	6899df7...	0.0246	1.1585	
357	08b9a6d...	0.742	0.958	1.7	198	6899df7...	0.0166	1.1297	
358	08b9a6d...	0.902	0.999	1.901	199	6899df7...	0.0136	1.0706	
...	...	...	...	...	...	...	...	...	...

359 rows × 4 columns

200 rows × 3 columns

Figure 6: An Example of Comparing Two *Pandas DataFrame* Outputs.

Most existing work on notebook reproducibility either overlooks image outputs or excludes them from their evaluation. For instance, Pimentel et al. [26] excluded the image outputs when applying normalizations on different types of outputs to compare execution results. This gap highlights the difficulty of using traditional deterministic or byte-wise methods, such as exact file comparison or Mean Squared Error (MSE), which can be overly sensitive to minor pixel-level variations that are perceptually insignificant.

To address these limitations, the SRI method employs the Structural Similarity Index Measure (SSIM) [41, 42] to compare image outputs. SSIM is a perceptual metric that evaluates image similarity based on structural information, contrast, and luminance. Unlike MSE, which penalizes even minor pixel differences equally, SSIM better aligns with human visual perception and focuses on the structural fidelity between two images (as shown in Figure 7). The algorithm produces a similarity score in the range [0, 1], where 1 indicates identical images and values closer to 0 indicate significant dissimilarity. SRI uses the SSIM implementation available in the scikit-image library [37] and considers the resulting similarity score as the reproducibility score for image outputs. By focusing on structural similarity rather than exact pixel-wise matches, this method provides a more robust and meaningful way to assess image output reproducibility in practical scenarios.

3.8 SRI for Miscellaneous Outputs

Being one of the most popular programming languages, Python supports numerous types of objects. As the SRI method largely depends on the data type of outputs, it also includes support for several miscellaneous types of outputs that are commonly encountered in Jupyter Notebooks:

- **Boolean values:** Outputs of type *bool* are compared directly. If the original and rerun outputs match exactly, the SRI score is set to 1; otherwise, it is set to 0.
- **Pandas series:** The SRI method reads and compares two *Pandas Series* outputs as *list* outputs.
- **Dictionary keys:** The SRI method converts any *dict_keys* outputs to *list* outputs and then apply the method for *list* comparison.

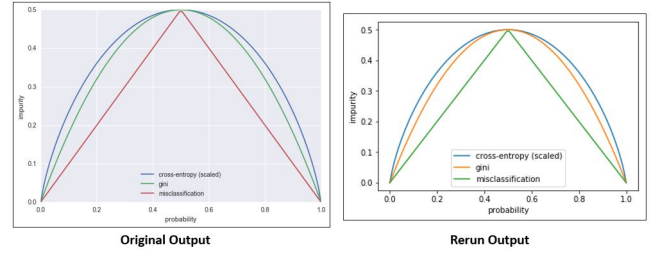


Figure 7: The outputs were generated using the `pandas.DataFrame.plot()` function. Cosmetic differences in the rerun output include changes in figure size, background color (gray vs. white), line colors (red, green, blue vs. green, orange, blue), the presence of a bounding box, and the absence of grid lines. Despite these superficial changes, the plot's structure and message remain consistent. While the traditional Mean Squared Error (MSE) is 18709.739, highlighting its oversensitivity, the Structural Similarity Index Measure (SSIM), used by SRI, yields a more meaningful and interpretable score of 0.666.

The following types of outputs exhibit inherently unstable behavior across notebook executions. They are compared using tailored logic to account for this variability, but their SRI scores are excluded from the overall notebook-level reproducibility score calculation:

- **Date and time:** Outputs of type *datetime.datetime* often vary based on system time, time zone, and other environmental factors. The SRI method identifies such objects with the help of regular expressions and compares them accordingly.
- **Memory addresses:** Many Python objects include memory addresses in their string representations, which can differ across executions. The SRI method uses regular expressions to detect such patterns for comparison.
- **Paths:** Outputs containing file or directory paths can differ due to variations in absolute versus relative path formatting, temporary directory names, or environment-specific file structures. The SRI method applies regular expressions to identify these outputs and string similarity to compare them.
- **Machine learning models:** Outputs from machine learning libraries such as *scikit-learn* [24] often include model initialization strings showing default hyperparameter settings. These outputs may vary between executions due to differences in default parameter values across library versions or environments. The SRI method uses regular expressions to detect common model signatures such as *KMeans*, *LogisticRegression*, *RandomForestClassifier*, *GradientBoostingClassifier*, and *Lasso*.

Any other type of Python objects that are not mentioned above are compared as string outputs. Similarly, in case of failures to parse the actual type of cell output objects, their plain text forms are compared as string outputs according to the SRI method.

4 Experiments

4.1 Experimental Setup

All the experiments for this work were performed on an 11th Gen Intel(R) Core(TM) i7-1185G7 CPU with a processing speed of 3.0 GHz and a RAM of 16 GB. The codes associated with the development, running, and testing of SRI were all written in Python using Jupyter Notebook version 6.4.12 with IPython version 7.31.1 running on Python version 3.9.13.

4.2 Dataset

For this experiment, 100 randomly selected Jupyter Notebooks and their rerun versions were used. The original notebooks are part of a larger corpus collected from GitHub in 2018, while the rerun versions were executed in 2023. Our focus in this study is on analyzing the results produced after rerunning the notebooks, rather than investigating the underlying factors that may cause execution errors or output variations. Since the primary goal of this work is to introduce and examine the effectiveness of a metric to assess reproducibility, we selected notebooks that executed successfully without runtime errors. In this context, we consider runnability a prerequisite for reproducibility. Similarly, to enable a straightforward one-to-one comparison of cell outputs, each rerun notebook was ensured to contain exactly the same number of cells as its corresponding original notebook. This structural consistency allowed us to align outputs from the original and rerun versions based on their respective cell positions. To evaluate the effectiveness of the proposed metric, we selected notebooks such that, across the dataset, there is at least one output change for each frequently used Python object type covered by SRI's type-specific comparison methods.

The dataset used in this experiment is publicly accessible on GitHub: <https://github.com/asm-shahadat/ACM-REP-25>.

4.3 Experimental Results and Discussion

As Figure 8 illustrates, a cell-level SRI result reports both a quantitative reproducibility score that ranges from 0 to 1 and several qualitative insights when the original and rerun outputs do not exactly match. Such cell-level SRI results are aggregated into a JSON structure to form the notebook-level SRI summary.

The above example represents a typical comparison of *NumPy* array outputs using SRI. While the arrays share the same data type, dimensions, size, and shape, they are not completely identical. However, an SRI score of approximately 0.47 indicates a partial match; a little fewer than half the elements are identical. This highlights how the SRI method moves beyond binary comparisons by incorporating structural and content-level analysis to provide a more detailed assessment of reproducibility for complex outputs. Additionally, the JSON structures storing the notebook-level SRI results are useful for storage, inspection, and automated analysis of reproducibility results across all cells in a notebook.

Figure 9 presents the notebook-level reproducibility scores for 100 notebooks as calculated by SRI. Among these, only two notebooks received a reproducibility score of zero, indicating that none of their outputs were successfully reproduced. In both cases, the *NumPy* array outputs in the rerun notebooks showed no

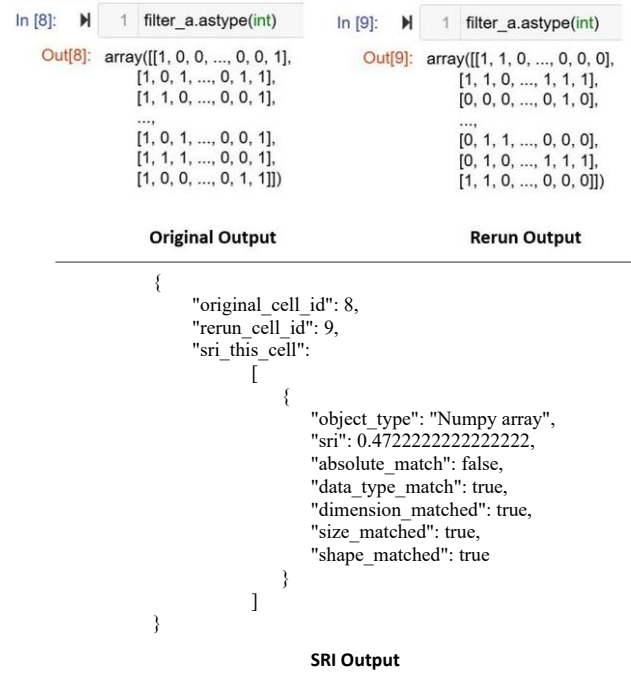


Figure 8: The *NumPy* array Outputs Changed due to Randomness. The Ellipses were Removed from the Cell Outputs Parsed by Using Text Processing, and then the Arrays were Compared. 17 out of 36 Elements Matched based on their Index Positions, and the SRI Output shows the reproducibility score calculated as 0.4722222222222222.

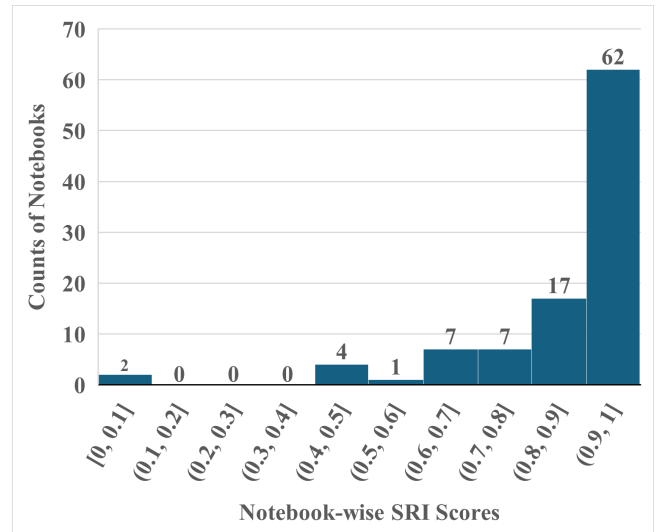


Figure 9: Histogram Showing the Overall Similarity-based Reproducibility Scores of 100 Jupyter Notebooks.

element-wise matches with the corresponding outputs in the original versions. On the other hand, 39 notebooks were found to be fully reproducible, with all outputs in the rerun versions identical to those in the originals. In total, 62 notebooks, including the fully reproducible ones, showed average reproducibility scores above 0.9, while 94 notebooks showed scores greater than 0.5.

To explore the effectiveness of the proposed metric, we further examined how the reproducibility of different types of outputs was perceived and evaluated by SRI across the 100 notebooks. This analysis provides insight into how well the metric captures reproducibility patterns at a finer granularity beyond notebook-level summaries.

Among different categories of outputs, we found that 462 out of 538 stream outputs across the 100 notebooks were absolutely reproduced. For the remaining outputs, partial reproducibility was determined using a string similarity algorithm, as described in Subsection 3.4, since stream outputs are plain texts and lack information about their specific Python object types. On the other hand, `display_data` outputs are predominantly images with the `image/png` MIME type. SRI computed scores for 117 such outputs using the SSIM-based method described in Subsection 3.7. Some of the broader categories of image output changes are as follows:

- **Changes due to randomness:** Many libraries, such as *Numpy*, offer options to generate random numbers, arrays, and statistical distributions. Often, these random objects are generated without specifying the seed value, which generates different outputs once the same code is rerun. For example, if a few random data points are created without any specific seed value to demonstrate how a clustering algorithm works, it will generate a completely different set of data points upon rerunning the same code. That will eventually change the clusters and cause changes in the rerun image outputs, visualizing those clusters. Other than the randomness caused by using *Numpy* objects without a specified seed value, other libraries may also cause similar issues. Another popular machine learning library in Python called *scikit-learn* [24] allows us to generate random samples for classification and clustering functions. If the `random_state` parameter of those functions is not used, then it will generate different data in different reruns. Figure 10 shows an example of image output changes due to randomness.
- **Changes due to different library versions:** Rerunning a notebook using a different version of the visualization library causes most of the image output changes. These types of changes include but are not limited to changes in figure size, colors, and tick styles and labels of charts. Figure 11 presents an example where the chart types changed due to using a different version of *Seaborn* [43] library to visualize the Iris dataset [11].
- **Changes due to data representation:** Image outputs may sometimes also change due to minor changes in data representation used for the visualization. Figure 12 shows one such example where the change in the data type of MNIST [18] Dataset imported from *keras.datasets* [7] caused changes in the image output.

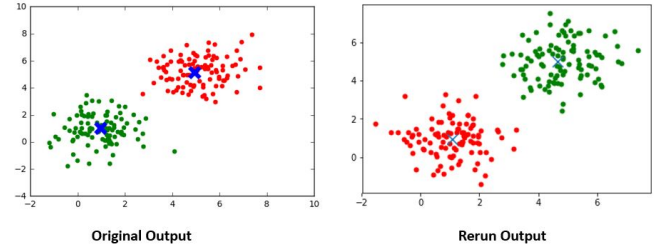


Figure 10: An Example of Image Output Changes due to Randomness (the Seed Value was not Specified for a *scikit-learn* Function to Generate Blobs of Points with a Gaussian Distribution). SSIM score: 0.66.

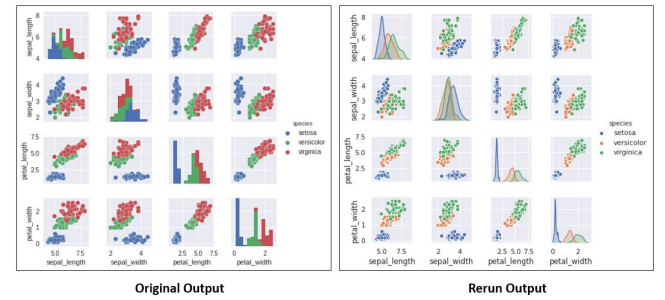


Figure 11: An Example of Image Output Changes due to Change in Library Versions (the Colormap and the Chart Type of the Diagonal Subplots Changed due to Running a Notebook using a Different Version of *Seaborn* Library). SSIM score: 0.56.

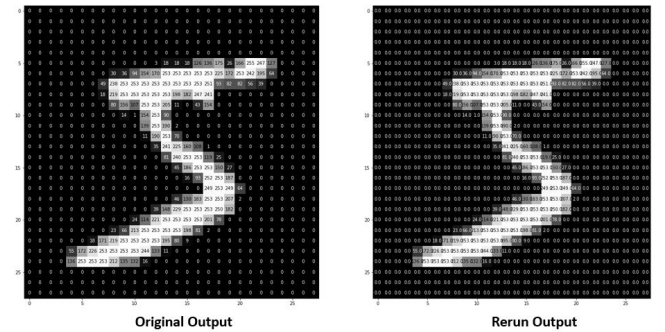


Figure 12: An Example of Image Output Changes due to Change in Data Representation (In the Rerun Image Output, the Pixel Values are Presented as *floats* unlike the *int* Values Presented in the Original Output. This Minor Change in the Rerun Output is Caused by the Change in the Underlying Data). SSIM score: 0.55.

Table 1 presents the experimental results for `execute_result` outputs, which are central to assessing cell-level reproducibility in Jupyter Notebooks. Out of 436 such outputs, 353 were found to be absolutely reproducible, meaning the rerun outputs were identical to the original ones. The remaining 83 outputs exhibited changes

Table 1: The table shows the number of objects compared, reproduced, and not reproduced based on traditional methods of assessing reproducibility, along with the number of partially reproduced objects identified by the Similarity-based Reproducibility Index (SRI). While traditional methods treat these objects as entirely non-reproducible, SRI reveals that some retain partial reproducibility, providing a finer-grained assessment.

Object type	Number of objects compared	Number of objects reproduced	Number of objects not reproduced	Number of objects partially reproduced as per SRI
int	54	44	10	0
float	27	25	2	0
list	31	28	3	3
tuple	17	15	2	1
str	26	24	2	1
dict	6	3	3	2
array	164	135	29	1
dataframe	70	50	20	19
others	41	29	12	9
Total	436	353	83	36

in the rerun notebooks and would be classified as non-reproducible under traditional reproducibility assessment methods, which rely solely on strict equality. However, this binary perspective fails to account for outputs that are not exactly identical yet still convey the same or similar computational intent. The Similarity-based Reproducibility Index (SRI) addresses this limitation by applying object-type-specific similarity metrics to capture partial reproducibility, providing a finer-grained and insightful assessment combining qualitative insights with the quantitative score.

SRI computes reproducibility scores for different types of Python objects using structural and semantic comparisons. For example, it calculates the proportion of matching elements for *list*, *tuple*, *dict*, *NumPy array*, and *Pandas DataFrame* outputs by considering index-aligned comparisons. For numeric data such as *int* and *float*, it takes into account acceptable differences in precision. Less common or undetermined object types are compared using string similarity algorithms like Jaro-Winkler, and image outputs are assessed using the SSIM metric. While image similarity scores can be difficult to interpret without established benchmarks, the underlying approach remains valid due to its structural awareness. Notably, even after excluding miscellaneous types from the `execute_result` outputs, SRI identified 27 outputs as partially reproduced out of 71 that were otherwise marked as non-reproducible by traditional methods. This demonstrates SRI's clear advantage in revealing subtle consistencies and offering a richer, similarity-aware perspective on computational reproducibility assessment.

SRI is particularly useful in collaborative scenarios where notebooks are shared and rerun. Often, environments are not shared, and thus, environments differ across collaborators, resulting in subtle differences. A partial reproducibility metric is helpful in quickly detecting and informing about the extent to

which notebook outputs, including intermediate outputs, were the same. They can also be used by organizations that maintain notebooks in non-container environments, but would like to evaluate the stability of computational experiments over time.

The limitations of SRI lie in the lack of interpretability and the absence of standardized thresholds for the similarity-based metrics used to compare different output types. The Jaro-Winkler algorithm, while effective for detecting minor typographical differences, primarily emphasizes character-level similarity and may overlook semantic or contextual differences in string outputs. As a result, SRI's reliance on this metric can lead to overestimated reproducibility scores when outputs differ in meaning despite being superficially similar. As future work, we plan to explore NLP-based techniques and text embedding models to assess semantic similarity, offering a more robust alternative for comparing string outputs that differ yet convey the same meaning.

Similarly, despite the effectiveness of quantifying perceptual similarity, the SSIM score used to compare image outputs lacks an established interpretability benchmark in the context of reproducibility analysis. Unlike numerical or textual outputs, image outputs present a unique challenge; changes that are visually minor or semantically insignificant may still result in low SSIM scores. As demonstrated in Figures 10, 11, and 12, even images that convey the same meaning or insight may score poorly due to changes in randomness, visualization libraries, or data representation. This makes the reproducibility assessment of image outputs more complex than other output types. Current similarity metrics like SSIM do not account for domain-specific semantics or the intent of the visualization. Consequently, while SSIM provides a useful quantitative measure, it is often insufficient for fully understanding whether the output remains functionally or semantically equivalent. These limitations point to a need for more robust, interpretable, and context-aware methods to assess the reproducibility of image outputs in future work.

5 Related Work

Jupyter Notebook, as a computational notebook, has received admiration from various domains. Johnson [15] mentioned it as a tool for literate programming [17]. Rowe et al. [29] discussed the potential of Jupyter Notebooks for scientific publications. They mentioned an initiative taken by a regional science-based journal called REGION, allowing authors to submit notebooks in a similar way to how they could submit a research paper. Beg et al. [3] studied tools of the Jupyter ecosystem and found the usage of IPyWidgets [14] useful for offering interactions. Figueiredo et al. [9] demonstrated the effectiveness of notebooks in increasing openness, reproducibility, and productivity in research by using them in ecology studies.

Being a modern implementation of the concept of Literate Programming [17], Jupyter Notebook, as well as tools developed on top of it, are commonly used for educational purposes. Casseau et al. [5] developed a Jupyter plug-in called Notebook Reproducibility Monitor (NoRM) to help students produce reproducible work. The tool was able to provide immediate feedback to its users regarding any practice that could affect the reproducibility of the code. Yin et al. [45] integrated notebooks

with High-Performance Computing (HPC) systems to build a framework for geospatial analytics. By satisfying the dependencies with Docker [19], they also found the tool effective after using it in teaching for a semester.

Pimentel et al. [25] conducted a large-scale study on analyzing the quality and reproducibility of 1.4 million Jupyter Notebooks collected from GitHub. They were able to rerun 20% of the notebooks without any exceptions to find only 4% reproducing the results. They extended their study after around two years to find that using different combinations of cell orders increased the reproducibility rate from 4.90% to 15.04%. Upon further exploration of 69 notebooks, they observed that the notebooks having more stars and forks on GitHub were educational and had markdown texts available. In a similar study with 1,000 Jupyter Notebooks, Wang et al. [39] found more than 70% of notebooks not reproducible. They identified the reasons, such as randomness, change in visualizations, and incorrect execution order of cells, causing poor reproducibility of results. Samuel et al. [33] were able to rerun 10% of the biomedical notebooks to find 5% of them reproducing results. They also built a tool named *ReproduceMeGit* [32] to analyze the reproducibility of Jupyter Notebooks by extracting dependency information from files such as *requirements.txt*, *setup.py*, and *pipfile* in a notebook repository.

Besides admiring the interactive environments offered by Jupyter Notebooks, researchers have also criticized some of its features. For example, Johnson [15], while praising the feature of easy documentation, pointed to the fact that Jupyter Notebook does not follow the best software engineering practices. Wang et al. [40] similarly accused notebooks of not ensuring quality by respecting the existing software engineering practices. They also drew attention to the strong need to analyze Jupyter Notebooks programmatically.

Several works found that the flexibility of notebooks in allowing the cells to run in any order is responsible for poor reproducibility. Wang et al. [39] accused the incorrect execution order of cells of affecting reproducibility. Casseau et al. [5] mentioned the unknown state of the value of a variable and the unknown order of actual cell executions as two reasons causing poor reproducibility of notebooks used in a classroom setting. Pimentel et al. [25] also suggested running the cells sequentially to ensure better reproducibility.

Rule et al. [30] identified the challenges in generating reproducible works. To overcome those challenges, they emphasized documenting the results using markdown cells. Several other works stated the significance of proper documentation in reproducibility. Schröder et al. [34] found a lack of documentation regarding the dependency to reconstruct environments for rerunning codes. Rule et al. [31], through an interview with 15 academic data analysts, found a lack in the documentation of analytical reasoning of the results in academic notebooks.

Fekete et al. [8], by exploring reproducibility in visualizations, provided a valuable insight that reproducibility is not exactly of the same meaning as correctness, as it is possible to reproduce incorrect results, yet failing to reproduce something can still be correct. On the other hand, Belz et al. [4] stated rerunnability as a

prerequisite of reproducibility while assessing the reproducibility of NLP results.

To improve the reproducibility of notebooks, it is important to know the factors that impact the reproducibility. Grotov et al. [12] highlighted the structural and stylistic differences between notebooks and traditional scripts. Stylistic features of notebooks are sometimes author-specific, as Pimentel et al. [25] mentioned the importance of importing the libraries at the top of notebooks. Oli et al. [23] used features extracted from notebooks to train machine learning models to assess their quality automatically. Bahaidarah et al. [2], through a survey with students and researchers, made a list of structural features of notebooks, such as indentation, blank lines, and comments, as helpful factors in increasing the readability of codes. Choetkiertikul et al. [6] performed an exploratory analysis of different characteristics of notebooks to find textual descriptions as an important factor in ensuring the quality of notebooks. Wang et al. [38] explored 80 top-ranked notebooks from Kaggle [16] competitions to spot differentiating practices between good and poor quality notebooks. They found that the well-documented quality notebooks not only use headlines but also document processes, future works, references, and explanations of results.

Despite these extensive efforts to understand and improve reproducibility in Jupyter Notebooks, most prior work has focused on binary assessments—determining whether notebooks rerun without error or produce identical outputs. However, they offer limited support for evaluating the degree of output consistency when differences do occur. This leaves a gap in assessing partial reproducibility, especially in scenarios where outputs are not identical but still semantically similar. Our work addresses this gap by introducing a similarity-based metric that quantifies how much outputs differ, enabling a more informative and practical evaluation of reproducibility.

6 Conclusion

This paper introduces the *Similarity-based Reproducibility Index (SRI)*, a novel metric for assessing reproducibility in Jupyter Notebooks. Unlike traditional binary approaches that simply detect whether outputs match or differ, SRI quantifies reproducibility using output type-specific similarity metrics, assigning a score in the range [0, 1]. It also provides qualitative insights to support a more detailed and meaningful evaluation of output consistency. By offering both cell-level and notebook-level assessments, SRI enables a more granular understanding of how rerun results compare to the original.

This work lays the foundation for using similarity-based metrics to assess computational reproducibility across broader scientific workflows.

Acknowledgments

This work is funded in part by the U.S. National Science Foundation (NSF) under award number 2516579.

Thanks to Dr. Hamed Alhoori and Dr. Reva Freedman, Associate Professors in the Department of Computer Science at Northern Illinois University, for their valuable feedback on earlier academic projects and the master's thesis related to this work.

References

- [1] Raza Ahmad, Naga Nithin Manne, and Tanu Malik. 2022. Reproducible notebook containers using application virtualization. In *2022 IEEE 18th International Conference on e-Science (e-Science)*. IEEE, 1–10.
- [2] Layan Bahaidarah, Ethan Hung, Andreas Francisco De Melo Oliveira, Jyotsna Penumaka, Lukas Rosario, and Ana Trisovic. 2022. Toward Reusable Science with Readable Code and Reproducibility. In *2022 IEEE 18th International Conference on e-Science (e-Science)*. 437–439.
- [3] Marijan Beg, Juliette Taka, Thomas Kluyver, Alexander Konovalov, Min Ragan-Kelley, Nicolas M Thiéry, and Hans Fangohr. 2021. Using Jupyter for Reproducible Scientific Workflows. *Comput. Sci. Eng.* 23, 2 (2021), 36–46.
- [4] Anya Belz. 2021. Quantifying Reproducibility in NLP and ML. *arXiv preprint arXiv:2109.01211* (Sept. 2021). arXiv:2109.01211 [cs.CL]
- [5] Christophe Casseau, Jean-Rémy Falleri, Xavier Blanc, and Thomas Degueule. 2021. Immediate Feedback for Students to Solve Notebook Reproducibility Problems in the Classroom. In *2021 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 1–5.
- [6] Morakot Choetkietikul, Apirak Hoonlor, Chaiyong Ragkhitwetsagul, Siripen Pongpaichet, Thanwadee Sunethanta, Tasha Sette Wong, Vacharavich Jiravatvanich, and Urisayar Kaewpichai. 2023. Mining the Characteristics of Jupyter Notebooks in Data Science Projects. *arXiv preprint arXiv:2304.05325* (April 2023). arXiv:2304.05325 [cs.SE]
- [7] François Chollet et al. 2015. Keras. <https://keras.io>.
- [8] Jean-Daniel Fekete, Juliana Freire, and Theresa-Marie Rhyne. 2020. Exploring Reproducibility in Visualization. *IEEE Comput. Graph. Appl.* 40, 5 (2020), 108–119.
- [9] Ludmilla Figueiredo, Cédric Scherer, and Juliano Sarmento Cabral. 2022. A Simple Kit to Use Computational Notebooks for More Openness, Reproducibility, and Productivity in Research. *PLoS Comput. Biol.* 18, 9 (Sept. 2022), e1010356.
- [10] Miloš Fišar, Ben Greiner, Christoph Huber, Elena Katok, Ali I Ozkes, and Management Science Reproducibility Collaboration. 2024. Reproducibility in Management Science. *Manage. Sci.* 70, 3 (March 2024), 1343–1356.
- [11] RA Fisher. 1936. Iris data set. UCI machine learning repository.
- [12] Konstantin Grotov, Sergey Titov, Vladimir Sotnikov, Yaroslav Golubev, and Timofey Bryksin. 2022. A Large-Scale Comparison of Python Code in Jupyter Notebooks and Scripts. In *Proceedings of the 19th International Conference on Mining Software Repositories*. 353–364.
- [13] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. 2020. Array Programming with NumPy. *Nature* 585, 7825 (Sept. 2020), 357–362. doi:10.1038/s41586-020-2649-2
- [14] IPyWidgets. 2024. IPyWidgets. <https://github.com/jupyter-widgets/ipywidgets>. Accessed: 2024-4-12.
- [15] Jeremiah W Johnson. 2020. Benefits and Pitfalls of Jupyter Notebooks in the Classroom. In *Proceedings of the 21st Annual Conference on Information Technology Education (Virtual Event, USA) (SIGITE '20)*. 32–37.
- [16] Kaggle. 2024. Kaggle: Your Machine Learning and Data Science Community. <https://www.kaggle.com/>. Accessed: 2024-4-12.
- [17] Donald Ervin Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (Jan. 1984), 97–111.
- [18] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [19] Dirk Merkel and Others. 2014. Docker: Lightweight Linux Containers for Consistent Development and Deployment. *Linux J.* 239, 2 (2014), 2.
- [20] James W Moody, Lisa A Keister, and Maria C Ramos. 2022. Reproducibility in the Social Sciences. *Annu. Rev. Sociol.* 48, 1 (July 2022), 65–85.
- [21] National Academies of Sciences, Engineering, and Medicine. 2019. *Reproducibility and Replicability in Science*. National Academies Press (US).
- [22] NumPy. 2024. `numpy.set_printoptions`. https://numpy.org/doc/stable/reference/generated/numpy.set_printoptions.html. Accessed: 2024-03-26.
- [23] Priti Oli, Rabin Banjade, Lasang Jimba Tamang, and Vasile Rus. 2021. Automated Assessment of Quality of Jupyter Notebooks Using Artificial Intelligence and Big Code. *The International FLAIRS Conference Proceedings* 34, 1 (April 2021).
- [24] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [25] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2019. A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. 507–517.
- [26] João Felipe Pimentel, Leonardo Murta, Vanessa Braganholo, and Juliana Freire. 2021. Understanding and Improving the Quality and Reproducibility of Jupyter Notebooks. *Empir. Softw. Eng.* 26, 4 (May 2021), 65.
- [27] Benjamin Ragan-Kelley, Carol Willing, F Akici, D Lippa, D Niederhut, and M Pacer. 2018. Binder 2.0-Reproducible, interactive, sharable environments for science at scale. In *Proceedings of the 17th Python in Science Conference*. F. Akici, D. Lippa, D. Niederhut, and M. Pacer, eds., 113–120.
- [28] Leonard Richardson. 2007. Beautiful Soup Documentation. (April 2007).
- [29] Francisco Rowe, Gunther Maier, Daniel Arribas-Bel, and Sergio J Rey. 2020. The Potential of Notebooks for Scientific Publication: Reproducibility, and Dissemination. *Reg. Anaesth.* 7, 3 (2020).
- [30] Adam Rule, Amanda Birmingham, Cristál Zuniga, Ilkay Altintas, Shih-Cheng Huang, Rob Knight, Niema Moshiri, Mai H Nguyen, Sara Brin Rosenthal, Fernando Pérez, and Peter W Rose. 2018. Ten Simple Rules for Reproducible Research in Jupyter Notebooks. *arXiv preprint arXiv:1810.08055* (Oct. 2018). arXiv:1810.08055 [cs.OH]
- [31] Adam Rule, Aurélien Tabard, and James D Hollan. 2018. Exploration and Explanation in Computational Notebooks. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*.
- [32] Sheeba Samuel and Birgitta König-Ries. 2021. ReproduceMeGit: A Visualization Tool for Analyzing Reproducibility of Jupyter Notebooks. In *Provenance and Annotation of Data and Processes*. 201–206.
- [33] Sheeba Samuel and Daniel Mietchen. 2024. Computational Reproducibility of Jupyter Notebooks from Biomedical Publications. *GigaScience* 13 (2024), giad113.
- [34] Max Schröder, Frank Krüger, and Sascha Spors. 2019. Reproducible Research is More than Publishing Research Artefacts: A Systematic Analysis of Jupyter Notebooks from Research Articles. *arXiv preprint arXiv:1905.00092* (April 2019). arXiv:1905.00092 [cs.DL]
- [35] Michael J Steventon, Christopher A-L Jackson, Matt Hall, Mark T Ireland, Marcus Munafo, and Kathryn J Roberts. 2022. Reproducibility in Subsurface Geoscience. *Earth Sci. Syst. Soc.* 2 (Sept. 2022).
- [36] The Pandas Development Team. 2020. *pandas-dev/pandas: Pandas*. doi:10.5281/zenodo.3509134
- [37] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulgong, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, Tony Yu, and the scikit-image contributors. 2014. scikit-image: image processing in Python. *PeerJ* 2 (6 2014), e453. doi:10.7717/peerj.453
- [38] April Yi Wang, Dakuo Wang, Jaimie Drozdal, Xuye Liu, Soya Park, Steve Oney, and Christopher Brooks. 2021. What Makes a Well-Documented Notebook? A Case Study of Data Scientists' Documentation Practices in Kaggle. In *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*.
- [39] Jiawei Wang, Tzu-Yang Kuo, Li Li, and Andreas Zeller. 2021. Assessing and Restoring Reproducibility of Jupyter Notebooks. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE '20)*. 138–149.
- [40] Jiawei Wang, Li Li, and Andreas Zeller. 2020. Better Code, Better Sharing: On the Need of Analyzing Jupyter Notebooks. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*. 53–56.
- [41] Zhou Wang and Alan C Bovik. 2009. Mean squared error: Love it or leave it? A new look at signal fidelity measures. *IEEE signal processing magazine* 26, 1 (2009), 98–117.
- [42] Zhou Wang, Alan Conrad Bovik, Hamid Rahim Sheikh, and Eero P Simoncelli. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.* 13, 4 (April 2004), 600–612.
- [43] Michael L. Waskom. 2021. seaborn: statistical data visualization. *Journal of Open Source Software* 6, 60 (2021), 3021. doi:10.21105/joss.03021
- [44] William E Winkler. 1990. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. (1990).
- [45] Dandong Yin, Yan Liu, Hao Hu, Jeff Terstriep, Xingchen Hong, Anand Padmanabhan, and Shaowen Wang. 2019. CyberGIS-Jupyter for reproducible and scalable geospatial analytics. *Concurr. Comput.* 31, 11 (June 2019), e5040.